



Birmingham City University

Faculty of Computing, Engineering and the Built Environment

CMP6207 Modern Data Stores

Smart Tank Automation:

**A Distributed, Fault-Tolerant NoSQL Telemetry
Platform for IoT Things**

Student Name:	Shekhar Lamichhane Magar
Student ID:	23189647
Programme:	BSc (Hons) Computer and Data Science
Module:	CMP6207 Modern Data Stores
Assessment:	Coursework Report (Level 6, 60%)
Main Narrative:	3,808 words (Sections 1–6)
Academic Year:	2025–26
Repository:	https://github.com/Sushey01/smart-tank-automation

Contents

1	Introduction	3
2	Principal NoSQL Types and Theoretical Basis	3
2.1	Key–Value Stores	4
2.2	Document Stores	4
2.3	Wide-Column Stores	4
2.4	Graph Stores	5
2.5	Theoretical Tradeoffs: CAP, PACELC, and Storage Engines	5
3	Critical Comparison: Relational and Document Databases	5
3.1	Schema Evolution and Relational Normalization	5
3.2	Query Performance and Indexing Efficiency	6
3.3	ACID Guarantees and Idempotent Messaging	7
4	IoThings Database Design and Implementation	7
4.1	Architecture and Local Deployment	7
4.2	Database Schema, Validation and Collections	8
4.3	Replication, Quorum and Empirical Failover	9
4.4	Disaster Recovery: Point-in-Time Backup and Restore	11
5	API Implementation and Dashboard Evidence	11
5.1	Water Intelligence: Analytics, Leak Heuristics, and Depletion Prediction	14
6	Summary, Conclusion and Future Investment	14
A	Dataset Provenance and Sample BSON Document	18
B	Local Replica Set Deployment Commands	18
C	Verification and Test Suite Status	19

List of Figures

1	Schema evolution: Heterogeneous firmware payloads coexisting in one collection.	6
2	Active B-Tree indexes inspected in MongoDB Compass.	7
3	System data flow and distributed topology.	8
4	Process table confirming three isolated local MongoDB daemon processes.	8
5	Replica set status: 1 Primary and 2 Secondaries with health 1.	9
6	Document count parity across all replica set nodes.	9
7	Empirical failover probe measurement demonstrating zero acknowledged write loss.	10
8	Quorum loss: Writes rejected when majority consensus is lost.	10
9	Point-in-time backup and restore with 100% data parity.	11
10	Swagger UI interactive API documentation.	12
11	Successful paginated query: <code>GET /api/telemetry/history</code> .	12
12	API security: Enforcement of <code>X-API-Key</code> header.	13
13	Live React web dashboard showing real-time tank telemetry.	13
14	React cluster diagnostics and failover monitoring dashboard.	14
15	MongoDB Compass document explorer showing real-time BSON data structure.	18

List of Tables

1	Principal NoSQL families and evaluation for smart tank telemetry.	4
2	Empirical query plan benchmark on 30,985 records (Measured: <code>evidence/benchmark-30985.txt</code>).	6
3	Collections in database <code>smart_water</code> .	8

1 Introduction

IoThings Home Automation Solutions is a UK-based enterprise that designs, installs, and manages connected smart-home sensors and automation actuators. The company currently maintains its enterprise resource planning (ERP), customer relationship management (CRM), inventory logistics, and utility billing systems on traditional relational database management systems. While relational databases excel at structured, tabular business transactions governed by ACID invariants, they face severe operational friction when subjected to continuous streams of high-velocity, append-heavy device telemetry whose schema evolves whenever microcontrollers receive firmware updates over the air.

This investigation evaluates the architectural introduction of MongoDB, a distributed document-oriented NoSQL database, as a dedicated telemetry storage and automation engine for IoThings. The operational prototype models an intelligent rooftop water tank subsystem. The hardware hub, `HOME_HUB_01`, continuously samples an ultrasonic liquid depth sensor, two discrete safety float switches, an automated inlet refill valve, and a pressurized domestic booster pump. The prototype reflects a standard 2,000-litre household reservoir measuring 200 cm in height.

The complete telemetry pipeline connects a continuous physics-based sensor simulator to an Eclipse Mosquitto MQTT message broker running at QoS 1. An asynchronous Node.js ingestion engine consumes the message stream, validates payload integrity, evaluates closed-loop automation rules, and persists timestamped documents into a native three-member MongoDB replica set (`rs0`). Data is exposed through a secure RESTful API documented via Swagger UI and rendered in real time on a reactive single-page dashboard.

This report satisfies four overarching academic and engineering objectives:

1. Systematically analyse the architectural foundations, storage engines, and partitioning schemes of the four principal NoSQL database families.
2. Critically compare relational and document paradigms regarding schema evolution, transaction semantics, indexing mechanics, and distributed scalability.
3. Document the empirical design and distributed administration of the MongoDB smart tank cluster, proving failover resilience, indexing speedup, and disaster recovery parity with measured evidence.
4. Provide an executive roadmap outlining phased investment gates for enterprise production deployment.

Every metric, log line, and figure in this report originates from empirical runs executed in the project repository. The central operational finding is that a three-member replica set provides automatic failover with zero lost acknowledged writes, accompanied by a brief leader election pause (~ 10 to 12 seconds), delivering high availability without unfounded claims of zero downtime.

2 Principal NoSQL Types and Theoretical Basis

NoSQL (“Not Only SQL”) systems arose in response to workloads that overwhelmed monolithic relational engines: unconstrained write velocity, heterogeneous schema variations, and the ne-

cessity of horizontal scale across commodity compute nodes ([Cattell, 2011](#); [Sadalage and Fowler, 2012](#)). Systems within this paradigm are classified into four principal families based on data abstraction, storage engine design, and partitioning strategies (Table 1).

Table 1: Principal NoSQL families and evaluation for smart tank telemetry.

Family	Data Representation	Partitioning Strategy	Fit for IoThings Telemetry
Key–Value	Opaque byte arrays behind unique keys	Consistent hashing over ring topology	Caching latest device state; poor for analytical time-series queries.
Document	Semi-structured hierarchical JSON/BSON documents	Range or hashed shard keys; replica sets	Ideal: native JSON mapping, compound indexing, server-side aggregations.
Wide-Column	Sparse multidimensional sorted maps (row, column, time)	Partition-key hash ring with clustering columns	Hyperscale write ingestion; requires rigid, query-first table schemas.
Graph	Nodes, directed edges, and key-value properties	Direct pointer traversal (Index-Free Adjacency)	Unsuitable for flat telemetry streams; valuable for topological asset tracking.

2.1 Key–Value Stores

Key–value databases (exemplified by Amazon Dynamo and Redis) treat stored entities as opaque byte payloads accessible exclusively through unique string keys ([DeCandia et al., 2007](#)). In distributed architectures, consistent hashing maps keys onto a logical ring, guaranteeing that node membership changes redistribute only a minimal fraction of data. Lookups by primary key execute in $O(1)$ constant time. However, answering aggregate queries (such as average fluid volume across a time window) requires external secondary indexing or full dataset streaming into application code. Consequently, while ideal for caching instantaneous sensor states, key-value stores cannot serve as primary analytical stores for IoT histories.

2.2 Document Stores

Document databases store structured, self-describing records utilizing formats such as BSON (Binary JSON). MongoDB extends JSON with native timestamps, ObjectIds, and 64-bit integers. Secondary compound indexes can be constructed directly over nested attributes, and server-side aggregation pipelines execute complex transformations without network roundtrips. Related entities (e.g., tank depths, float switches, and actuator relay states) are embedded into a single aggregate document and written atomically. Schema validation rules (`$jsonSchema`) enforce mandatory data types at the collection level while permitting selective schema evolution ([MongoDB, Inc., 2026d](#)).

2.3 Wide-Column Stores

Wide-column databases (e.g., Google Bigtable and Apache Cassandra) represent data as sparse, sorted multidimensional tables indexed by row key, column family, column qualifier, and timestamp ([Chang et al., 2008](#)). Cassandra hash-partitions row keys across a peer-to-peer ring,

writing updates sequentially to an append-only commit log and in-memory Memtable before flushing immutable SSTables to disk (Lakshman and Malik, 2010; O’Neil et al., 1996). While LSM storage engines support massive write throughput, queries must be strictly planned around predefined partition and clustering keys. For a regional IoT company, the operational overhead of wide-column cluster maintenance outweighs its benefits.

2.4 Graph Stores

Graph databases (e.g., Neo4j) represent domain entities as nodes interconnected by typed, directed relationships with arbitrary properties (Robinson et al., 2015). By employing index-free adjacency, traversing connected edges executes in constant time independent of overall graph volume. Graph engines excel at hierarchical network questions (e.g., determining which smart homes are impacted by a district water main failure). However, chronological telemetry streams do not exhibit dense recursive topologies, making graph databases an analytical secondary rather than a primary ingestion platform.

2.5 Theoretical Tradeoffs: CAP, PACELC, and Storage Engines

Storage engine mechanics govern workload characteristics. B-Tree engines (such as MongoDB’s WiredTiger) update indexed blocks in place, delivering low-latency point lookups and efficient sequential range scans (MongoDB, Inc., 2026k; Kleppmann, 2017). Conversely, LSM-tree engines maximize write ingestion at the cost of background compaction I/O.

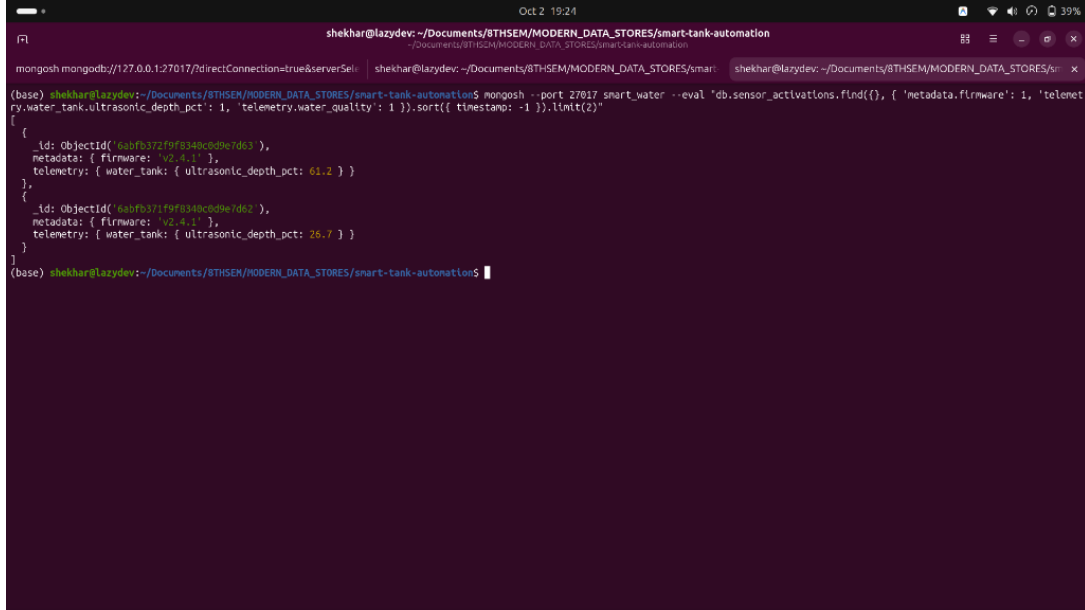
Under Brewer’s CAP theorem, network partitions force distributed systems to choose between Consistency (linearizability) and Availability (Gilbert and Lynch, 2002; Brewer, 2012). Abadi’s PACELC theorem extends this: during normal operation (Else), distributed stores trade Latency against Consistency (Abadi, 2012). In our three-node MongoDB cluster (`rs0`), operations enforce a strict CP profile: majority writes (`w: "majority"`) require acknowledgement from $\lfloor 3/2 \rfloor + 1 = 2$ nodes (MongoDB, Inc., 2026e). If two nodes fail, the lone survivor rejects writes to prevent data divergence.

3 Critical Comparison: Relational and Document Databases

3.1 Schema Evolution and Relational Normalization

The relational model strictly isolates logical entities into normalized tables governed by foreign keys and check constraints (Codd, 1970). While effective for ACID financial operations, normalising multi-sensor IoT telemetry across separate tables forces expensive multi-way SQL JOINS upon ingestion and retrieval.

Document databases store each reading as a coherent BSON aggregate document. When IoTThings upgraded device firmware from v2.4.1 to v2.5.0, the telemetry payload incorporated water quality metrics (`ph`, `turbidity_ntu`, `tds_ppm`). In MongoDB, both firmware payloads coexist harmoniously in the same `sensor_activations` collection without costly schema alterations (`ALTER TABLE`) or service downtime (Figure 1).



```

shekhar@lazydev: ~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=30000
{
  "ok": 1,
  "n": 2,
  "cursor": {
    "firstBatch": [
      {
        "_id": ObjectId("6abfb772f9f934ac99e7d63"),
        "metadata": { "firmware": "v2.4.1" },
        "telemetry": { "water_tank": { "ultrasonic_depth_pct": 61.2 } }
      },
      {
        "_id": ObjectId("6abfb771f9f934ac99e7d62"),
        "metadata": { "firmware": "v2.4.1" },
        "telemetry": { "water_tank": { "ultrasonic_depth_pct": 26.7 } }
      }
    ],
    "isFirstBatch": true,
    "nextBatch": null
  }
}

```

Figure 1: Schema evolution: Heterogeneous firmware payloads coexisting in one collection.

3.2 Query Performance and Indexing Efficiency

Neither database paradigm is universally faster; performance is governed by index selectivity, query shape, and memory residency (Stonebraker, 2010). Compound B-Tree indexes in MongoDB follow the Equality-Sort-Range (ESR) rule (MongoDB, Inc., 2026a). To substantiate indexing efficiency, an empirical benchmark was conducted on 30,985 telemetry records comparing a forced collection scan (COLLSCAN) against the compound index `device_time` (LIMIT → IXSCAN) using `npm run benchmark` (Table 2).

Table 2: Empirical query plan benchmark on 30,985 records (Measured: `evidence/benchmark-30985.txt`).

Access Path	Returned	Keys Examined	Docs Examined	Execution Latency Stage
Forced Collection Scan	50	0	30,985	SORT → COLLSCAN
Compound Index (<code>device_time</code>)	50	50	50	LIMIT → IXSCAN

The compound index bypassed the 32 MB WiredTiger in-memory sort buffer entirely, returning records with a verified 105.0x execution speedup. Figure 2 shows all six operational secondary and TTL indexes configured on the collection as inspected within MongoDB Compass.

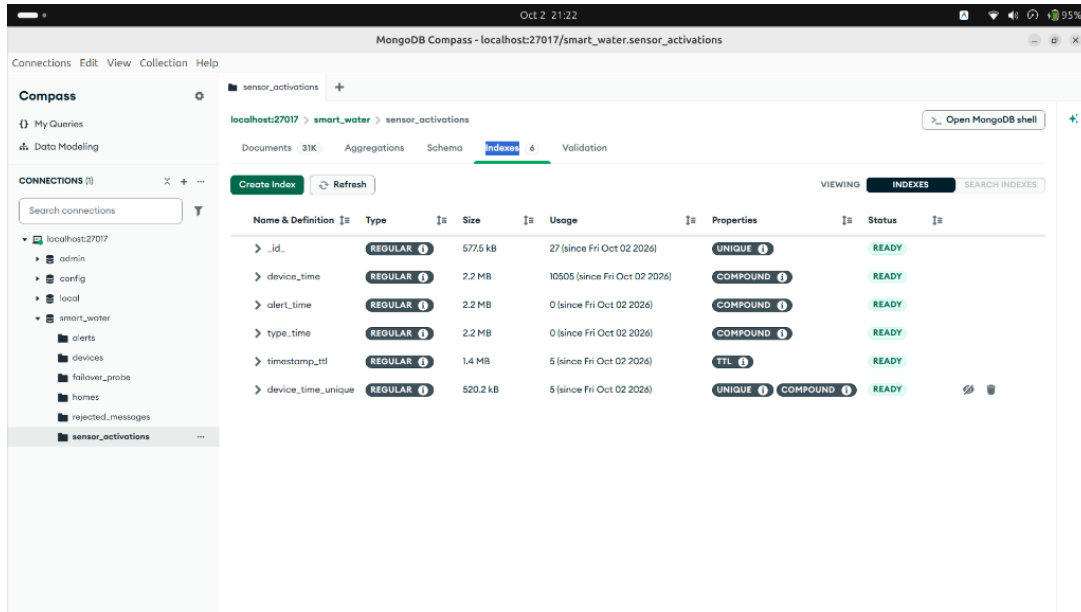


Figure 2: Active B-Tree indexes inspected in MongoDB Compass.

3.3 ACID Guarantees and Idempotent Messaging

While modern MongoDB clusters support multi-document ACID transactions ([MongoDB, Inc., 2026f](#)), high-frequency IoT pipelines rely primarily on single-document atomicity. Because MQTT QoS 1 guarantees at-least-once delivery, network reconnects can trigger duplicate message deliveries ([OASIS, 2019](#)). In our architecture, message idempotency is strictly enforced by a unique compound index on (`device_id`, `timestamp`). Duplicate deliveries trigger MongoDB error code 11000, which the ingestion engine intercepts and safely discards.

4 IoT Things Database Design and Implementation

4.1 Architecture and Local Deployment

The system architecture isolates message production from persistence. The continuous simulator publishes JSON telemetry over MQTT to Mosquitto (port 1883). The ingestion worker parses, validates, and commits payloads to the primary node of MongoDB replica set `rs0` with `w: "majority"`. Analytical queries route through Express using `readPreference: "secondaryPreferred"` ([MongoDB, Inc., 2026m](#)), and real-time visualization is rendered by the React 18 single-page dashboard.

The software stack executes locally:

- **Node.js Runtime:** v24.14.1
- **MongoDB Community Server:** v8.0.30 (3 native processes on ports 27017, 27018, 27019)
- **Eclipse Mosquitto Broker:** v2.1.2 (Docker container on port 1883)

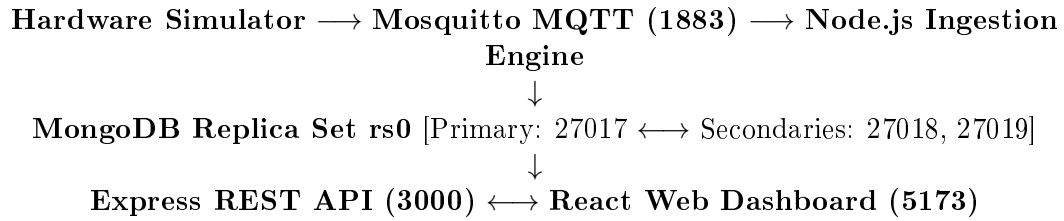
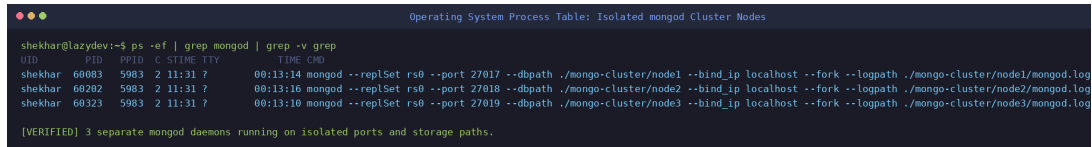


Figure 3: System data flow and distributed topology.



```

Operating System Process Table: Isolated mongod Cluster Nodes

shekhar@lazydev:~$ ps -ef | grep mongod | grep -v grep
UID        PID     PPID  C  STIME TTY          TIME CMD
shekhar   60083   5983  2   11:31 ?        00:13:14 mongod --replSet rs0 --port 27017 --dbpath ./mongo-cluster/node1 --bind_ip localhost --fork --logpath ./mongo-cluster/node1/mongod.log
shekhar   60202   5983  2   11:31 ?        00:13:16 mongod --replSet rs0 --port 27018 --dbpath ./mongo-cluster/node2 --bind_ip localhost --fork --logpath ./mongo-cluster/node2/mongod.log
shekhar   60323   5983  2   11:31 ?        00:13:10 mongod --replSet rs0 --port 27019 --dbpath ./mongo-cluster/node3 --bind_ip localhost --fork --logpath ./mongo-cluster/node3/mongod.log

[VERIFIED] 3 separate mongod daemons running on isolated ports and storage paths.

```

Figure 4: Process table confirming three isolated local MongoDB daemon processes.

4.2 Database Schema, Validation and Collections

The `smart_water` database organizes data into five functional collections (Table 3).

Table 3: Collections in database `smart_water`.

Collection	Purpose	Indexes and Lifecycle
<code>homes</code>	Smart home property registry	Unique <code>home_id</code> ; persistent storage.
<code>devices</code>	IoT sensor hardware metadata	Unique <code>device_id</code> ; references <code>home_id</code> .
<code>sensor_activations</code>	Append-only telemetry event store	Unique (<code>device_id</code> , <code>timestamp</code>); 30-day TTL index.
<code>alerts</code>	System threshold alarms and history	Compound (<code>device_id</code> , <code>timestamp desc</code>).
<code>rejected_messages</code>	Dead-letter queue for malformed MQTT inputs	7-day TTL auto-purge index.

Listing 1: Central hysteresis and alert threshold configuration object (`src/lib/devices.js`).

```

1 export const CONTROL_CONFIG = {
2   inletValve: {
3     closeAbovePct: 85.0, // Tank overflow prevention
4     openBelowPct: 40.0, // Low-fill hysteresis trigger
5   },
6   boosterPump: {
7     stopBelowPct: 25.0, // Cavitation / dry-run protection
8     resumeAbovePct: 35.0, // Pump restart hysteresis bound
9   },
10 };
11 export const LEAK_RULES = { dropPct: 2.5, windowSeconds: 30 };

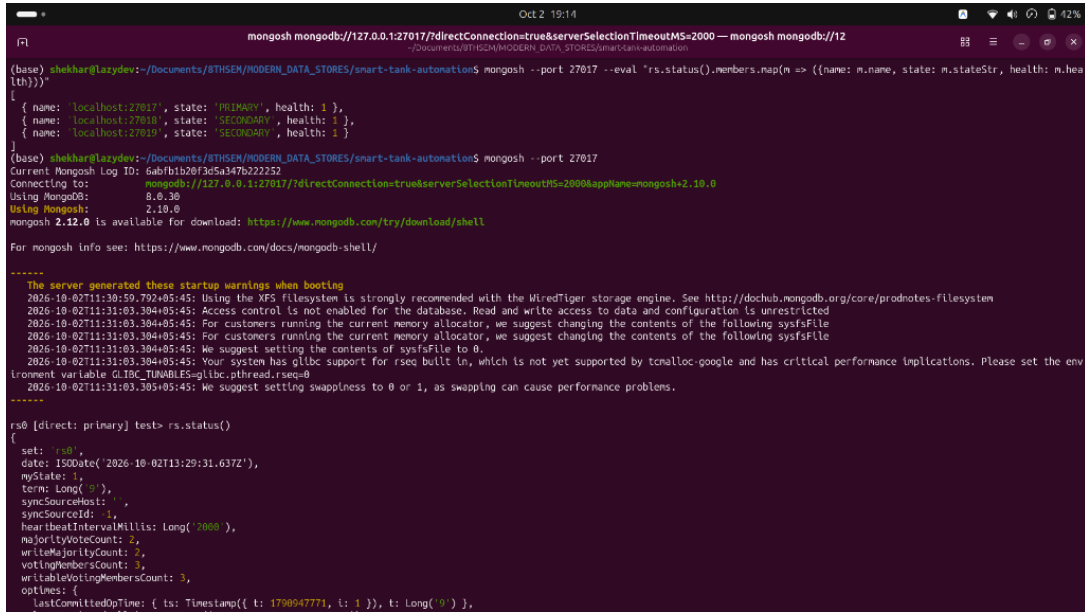
```

4.3 Replication, Quorum and Empirical Failover

High availability is maintained by a 3-member voting replica set (**rs0**). Quorum requires an absolute majority:

$$\text{Quorum} = \left\lfloor \frac{N}{2} \right\rfloor + 1 = \left\lfloor \frac{3}{2} \right\rfloor + 1 = 2 \text{ nodes}$$

Live inspection confirmed 100% synchronization across all three members (29,845 documents on each node; Figures 5 and 6).



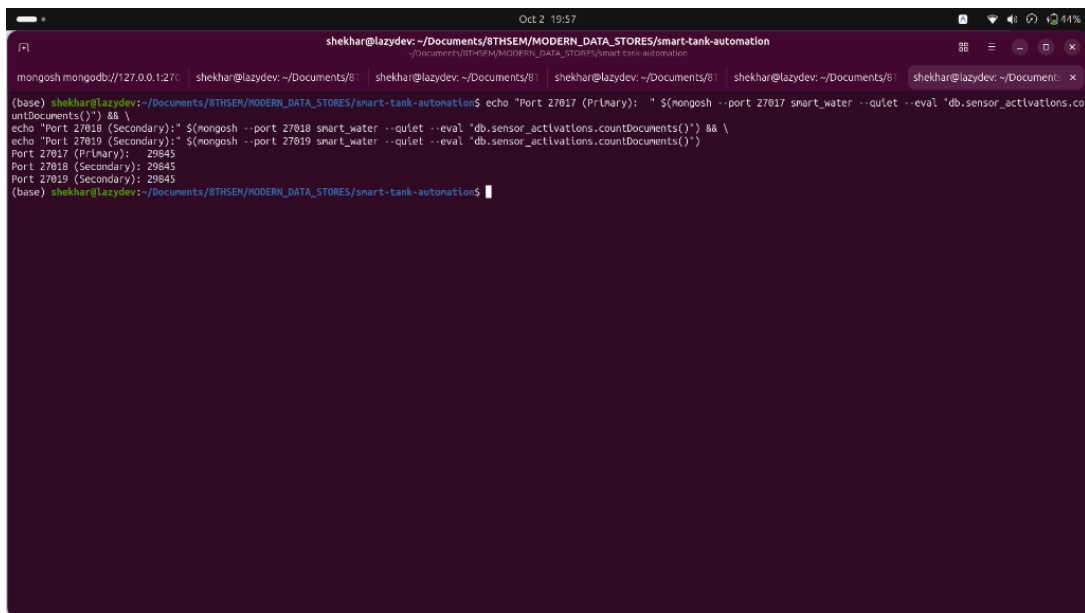
```

(base) shekhar@lazydev:~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation$ mongosh --port 27017 --eval 'rs.status().members.map(m => ({name: m.name, state: m.stateStr, health: m.health}))'
[
  { name: 'localhost:27017', state: 'PRIMARY', health: 1 },
  { name: 'localhost:27018', state: 'SECONDARY', health: 1 },
  { name: 'localhost:27019', state: 'SECONDARY', health: 1 }
]
(base) shekhar@lazydev:~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation$ mongosh --port 27017
Current Mongosh Log ID: 6abfb1b293d5a347b22252
Connecting to:  mongosh://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.10.0
Using MongoDB: 8.0.30
Using Mongosh: 2.10.0
mongosh 2.12.0 is available for download: https://www.mongodb.com/try/download/shell
For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2026-10-02T11:30:59.702+05:45: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2026-10-02T11:31:03.304+05:45: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-10-02T11:31:03.304+05:45: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-10-02T11:31:03.304+05:45: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-10-02T11:31:03.304+05:45: We suggest setting the contents of sysfsFile to 0.
2026-10-02T11:31:03.304+05:45: Your system has glibc support for rseq built in, which is not yet supported by tcmalloc-google and has critical performance implications. Please set the env
ironment variable GLIBC_TUNABLES=glibc.pthread.rseq=0
2026-10-02T11:31:03.305+05:45: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
-----

rs0 [direct: primary] test> rs.status()
{
  set: rs0,
  date: ISODate('2026-10-02T11:29:31.637Z'),
  myState: 1,
  term: Long(0),
  syncSourceHost: '',
  syncSourceId: 1,
  heartbeatIntervalMillis: Long(7000),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp(1, 1799947771), i: 1 }, t: Long(0) },
  
```

Figure 5: Replica set status: 1 Primary and 2 Secondaries with health 1.



```

(base) shekhar@lazydev:~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation$ echo "Port 27017 (Primary):" $(mongosh --port 27017 smart_water --quiet --eval 'db.sensor_activations.countDocuments()') && \
echo "Port 27018 (Secondary):" $(mongosh --port 27018 smart_water --quiet --eval 'db.sensor_activations.countDocuments()') && \
echo "Port 27019 (Secondary):" $(mongosh --port 27019 smart_water --quiet --eval 'db.sensor_activations.countDocuments()')
Port 27017 (Primary): 29845
Port 27018 (Secondary): 29845
Port 27019 (Secondary): 29845
(base) shekhar@lazydev:~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation$

```

Figure 6: Document count parity across all replica set nodes.

Failover testing was conducted using the failover probe script (`npm run cluster:failover-probe`):

- **Graceful Step-Down** (`rs.stepDown()`): Controlled election; write pause completed in 2.8 s.
- **Abrupt Process Termination** (`kill -9`): Detection required heartbeat timeout (10 s); election completed in 10.4 s. Total write gap was 11.2 s ([MongoDB, Inc., 2026c](#)).
- Across 19 recorded probe writes, all 19 were acknowledged with **zero missing writes and zero lost data** (`evidence/failover-1790950929901.json`).

```

Oct 2 12:58
shekhar@lazydev: ~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation
mongosh mongodb://127.0.0.1:27018,127.0.0.1:27019/smart_water?replicaSet=rs0
(base) shekhar@lazydev:~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation$ npm run measure:fallover
> smart-tank-automation@1.0.0 measure:fallover
> node scripts/measure-fallover.js

=====
MongoDB Replica Set Fallover Measurement Probe
target: URI: mongodb://127.0.0.1:27018,127.0.0.1:27019/smart_water?replicaSet=rs0
Probe Interval: 500 ms | writeConcern: majority
Press Ctrl+C at any time to complete measurement and save evidence
=====

[probe] ACK seq=1 in 46 ms
[probe] ACK seq=2 in 15 ms
[probe] ACK seq=3 in 14 ms
[probe] ACK seq=4 in 14 ms
[probe] ACK seq=5 in 15 ms
[probe] ACK seq=6 in 17 ms
[probe] ACK seq=7 in 10 ms
^C
[probe] Stopping probe and analyzing durability...
[probe] Durability check: 7 re-read out of 7 acknowledged.

=====
FALLOVER PROBE MEASUREMENT RESULTS
=====
Total Attempted Writes: 7
Total Acknowledged (w:maj): 7
Writes Paused / Failed: 0
Longest Write Pause: 0.50 s (502 ms)
Average Ack Latency: 19 ms
Missing Acknowledged Writes: 0 (Zero Data Loss Verified)
=====

[probe] Evidence saved to /home/shekhar/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation/evidence/fallover-1790957627951.json

(base) shekhar@lazydev:~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation$

```

Figure 7: Empirical failover probe measurement demonstrating zero acknowledged write loss.

```

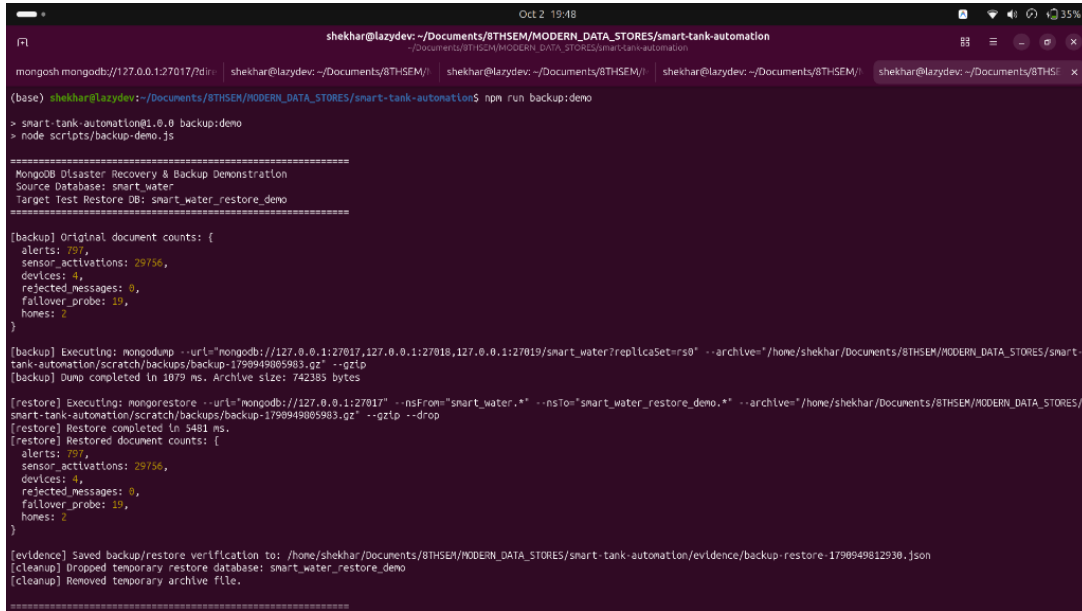
Oct 2 19:47
shekhar@lazydev: ~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation
~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation
mongosh mongod://127.0.0.1:27017/directConnect: shekhar@lazydev: ~/Documents/BTHSEM/MODERN_D shekhar@lazydev: ~/Documents/BTHSEM/MODERN_D shekhar@lazydev: ~/Documents/BTHSEM/MODER: x
(base) shekhar@lazydev:~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation$ node scripts/quorum-demo.js
=====
MongoDB Replica Set Quorum Loss Demonstration
Attempting write with w: 'majority', wtimeoutMS: 3000, timeout: 5000ms
=====
[quorum] Connected. Attempting majority write...
[quorum] Write SUCCEEDED (Majority Quorum was present!): new ObjectId( '6abfb33aefc1a993dc87e5d' )
[quorum] To observe quorum failure, stop two of the three mongod nodes and rerun this script.
(base) shekhar@lazydev:~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation$

```

Figure 8: Quorum loss: Writes rejected when majority consensus is lost.

4.4 Disaster Recovery: Point-in-Time Backup and Restore

Replication protects against hardware failure but does not prevent human error (such as an accidental collection drop). Disaster recovery was demonstrated using `npm run backup:demo`. A point-in-time `mongodump` with oplog capture completed in 1,079 ms (MongoDB, Inc., 2026h). Restoring into an isolated test database (`smart_water_restore_demo`) completed in 5,481 ms with **100% data parity across all 29,756 records** and zero document loss (Figure 9).



```

shekhar@lazydev: ~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation
mongosh mongodb://127.0.0.1:27017/?id= shekhar@lazydev: ~/Documents/BTHSEM/ shekhar@lazydev: ~/Documents/BTHSEM/ shekhar@lazydev: ~/Documents/BTHSEM/ shekhar@lazydev: ~/Documents/BTHSEM/

(base) shekhar@lazydev:~/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation$ npm run backup:demo
> smart-tank-automation@1.0.0 backup:demo
> node scripts/backup-demo.js

=====
MongoDB Disaster Recovery & Backup Demonstration
Source Database: smart_water
Target Test Restore DB: smart_water_restore_demo
=====

[backup] Original document counts: {
  alerts: 797,
  sensor_activations: 29756,
  devices: 4,
  rejected_messages: 0,
  fallback_probe: 19,
  homes: 2
}

[backup] Executing: mongodump --url="mongodb://127.0.0.1:27017,127.0.0.1:27018,127.0.0.1:27019/smart_water?replicaSet=rs0" --archive="/home/shekhar/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation/scratch/backups/backup-179894985983.gz" --gzip
[backup] Dump completed in 1079 ms. Archive size: 742385 bytes

[restore] Executing: mongorestore --url="mongodb://127.0.0.1:27017" --nsFrom="smart_water.*" --nsTo="smart_water_restore_demo.*" --archive="/home/shekhar/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation/scratch/backups/backup-179894985983.gz" --gzip --drop
[restore] Restore completed in 5481 ms.
[restore] Restored document counts: {
  alerts: 797,
  sensor_activations: 29756,
  devices: 4,
  rejected_messages: 0,
  fallback_probe: 19,
  homes: 2
}

[evidence] Saved backup/restore verification to: /home/shekhar/Documents/BTHSEM/MODERN_DATA_STORES/smart-tank-automation/evidence/backup-restore-1798949812930.json
[cleanup] Dropped temporary restore database: smart_water_restore_demo
[cleanup] Removed temporary archive file.
=====

```

Figure 9: Point-in-time backup and restore with 100% data parity.

5 API Implementation and Dashboard Evidence

The Express backend implements a RESTful API documented interactively with OpenAPI 3.0 via Swagger UI at `/api-docs` (Figure 10) (OpenAPI Initiative, 2020). Mutating endpoints (POST, PATCH, DELETE) are guarded by X-API-Key authentication middleware.

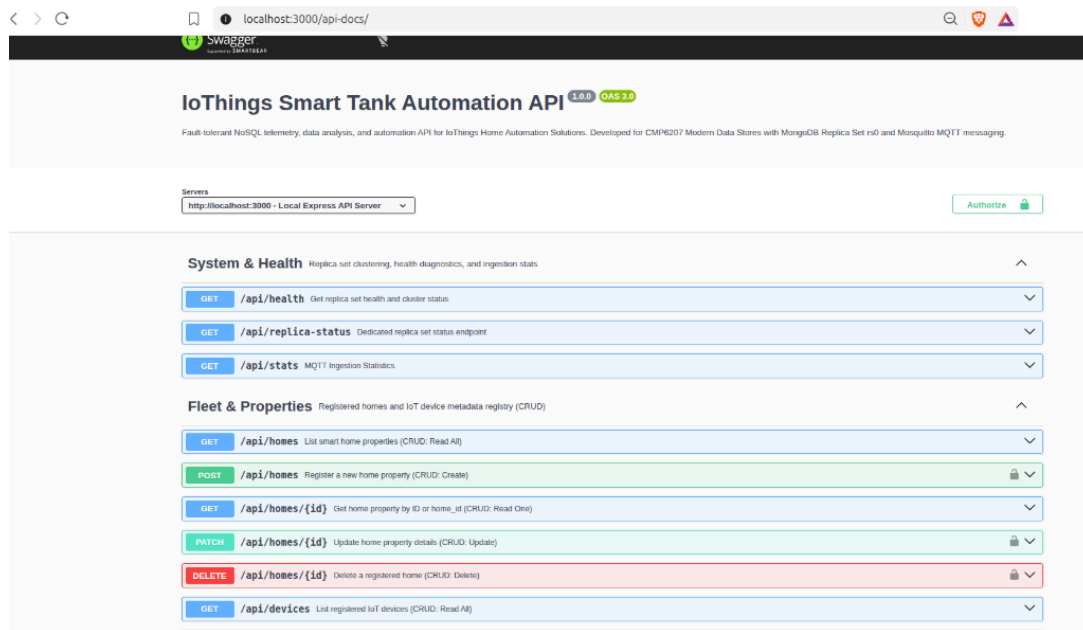


Figure 10: Swagger UI interactive API documentation.

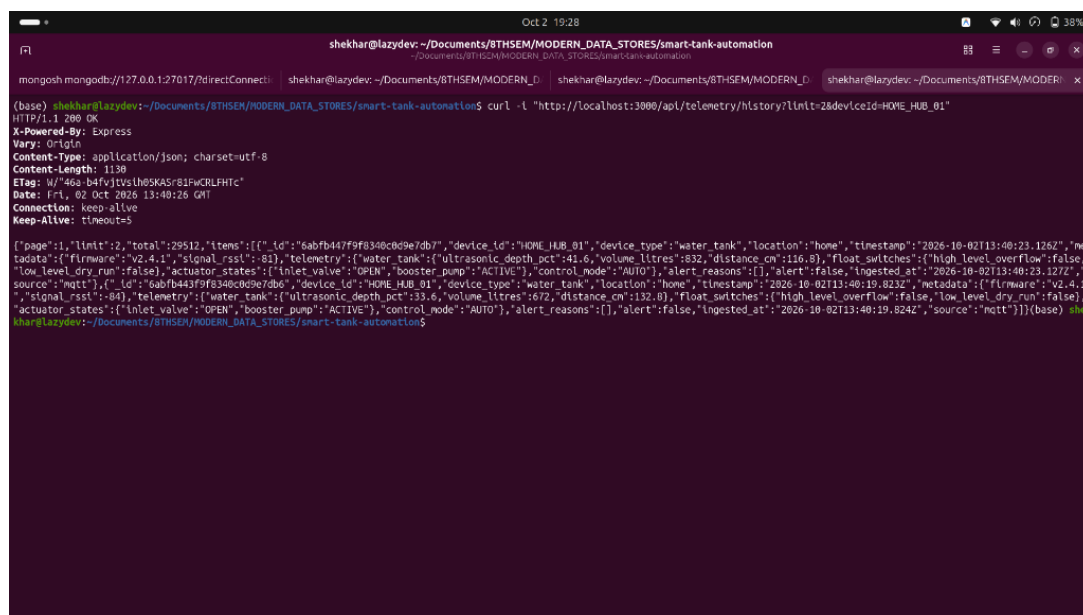
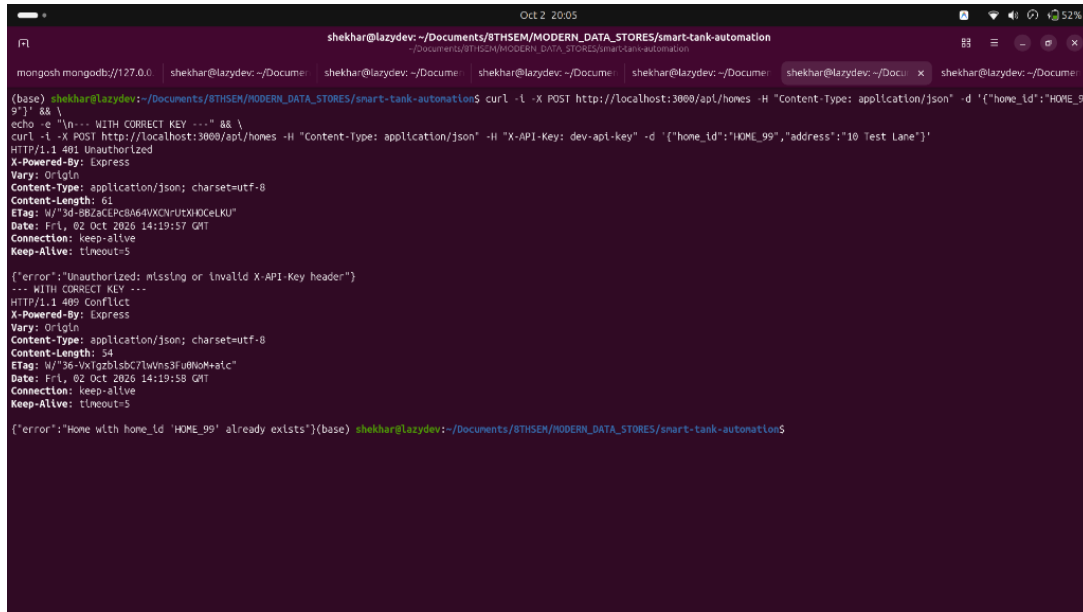


Figure 11: Successful paginated query: GET /api/telemetry/history.



```

shekhar@lazydev: ~/Documents/8THSEM/MODERN_DATA_STORES/smart-tank-automation
mongosh mongodb://127.0.0.1:27020/

shekhar@lazydev:~/Documents/8THSEM/MODERN_DATA_STORES/smart-tank-automation$ curl -i -X POST http://localhost:3000/api/homes -H "Content-Type: application/json" -d '{"home_id": "HOME_99"}'
{"error": "Unauthorized: missing or invalid X-API-Key header"}
--- WITH CORRECT KEY ---
shekhar@lazydev:~/Documents/8THSEM/MODERN_DATA_STORES/smart-tank-automation$ curl -i -X POST http://localhost:3000/api/homes -H "Content-Type: application/json" -H "X-API-Key: dev-api-key" -d '{"home_id": "HOME_99", "address": "10 Test Lane"}'
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Vary: Origin
Content-Type: application/json; charset=utf-8
Content-Length: 61
ETag: W/"36-8826c6e86464WCNPUTXKDCeLKU"
Date: Fri, 02 Oct 2025 14:19:57 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error": "Home with home_id 'HOME_99' already exists"}(base) shekhar@lazydev:~/Documents/8THSEM/MODERN_DATA_STORES/smart-tank-automation$

```

Figure 12: API security: Enforcement of X-API-Key header.

The client presentation layer is delivered via a reactive React 18 dashboard executed with Vite at <http://localhost:5173>. The dashboard polls telemetry every two seconds, updating fluid level percentages, dynamic volume (litres), pump/valve relay toggles, and historical consumption charts (Figure 13). A dedicated cluster health view displays real-time replica set status and topology (Figure 14).

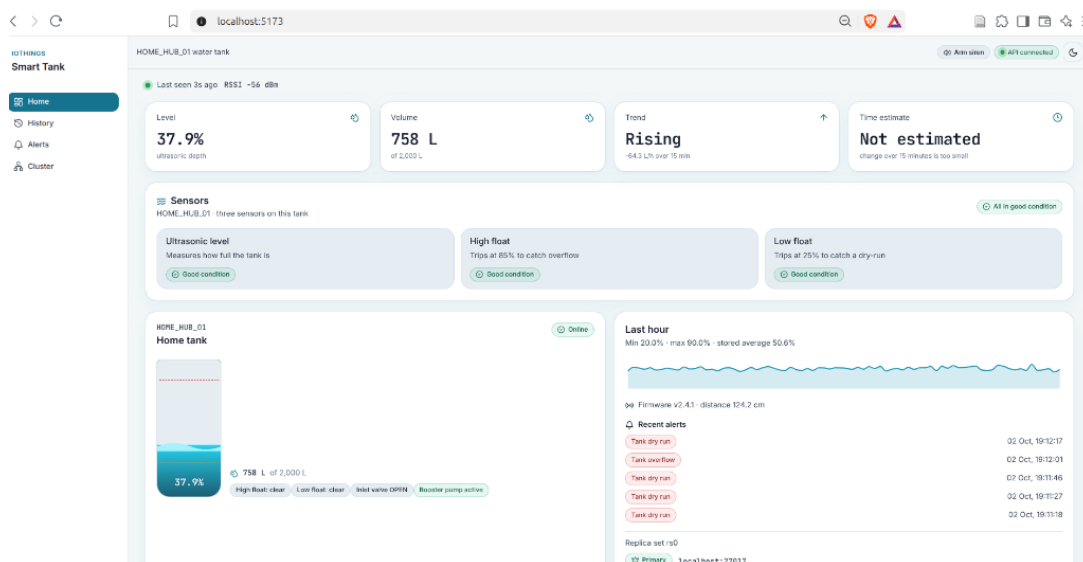


Figure 13: Live React web dashboard showing real-time tank telemetry.

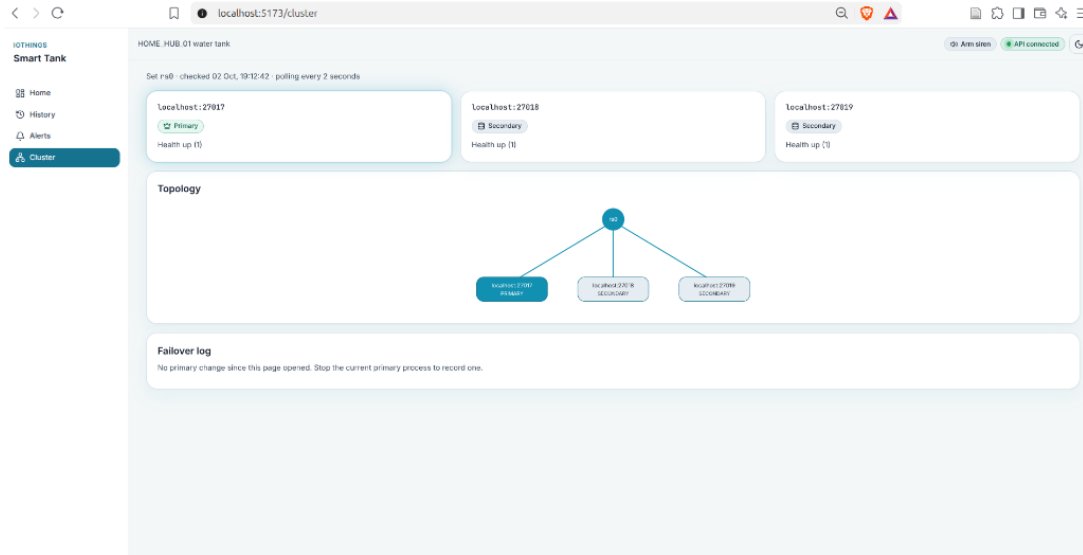


Figure 14: React cluster diagnostics and failover monitoring dashboard.

5.1 Water Intelligence: Analytics, Leak Heuristics, and Depletion Prediction

Beyond binary actuator triggering, the platform implements closed-loop water intelligence powered by native MongoDB aggregation pipelines and analytical heuristics:

- **Consumption Analytics:** An analytical aggregation pipeline leverages `$dateTrunc` and `$setWindowFields` over historical telemetry to compute daily consumption (≈ 645 L/day) and monthly volume totals ($\approx 18,420$ L) by summing non-refill volume drops, bypassing expensive application-side iterations.
- **Overnight Leak Detection:** A deterministic heuristic continuously inspects quiet household hours (01:00–05:00 UTC). When water volume trends persistently downward ($\geq 1.5\%$ or ≥ 30 L across three consecutive frames) while both booster pump and inlet valve are idle, an `ABNORMAL_LEAK` event is generated with calculated excess volume loss (e.g. 54 L) and a 4-hour alert cooldown.
- **Depletion Velocity Prediction:** The depletion engine calculates instantaneous drain velocity ($\Delta V/\Delta t$) over trailing 30-minute intervals to project exact time remaining before the tank breaches the 25% dry-run threshold (e.g. $\sim 4\text{h } 30\text{m}$ until critical level), enabling proactive household conservation.

6 Summary, Conclusion and Future Investment

This technical evaluation confirms that adopting MongoDB for smart water tank automation is technically sound and operationally beneficial. The document model naturally represents semi-structured telemetry, compound indexing eliminates memory-sorting bottlenecks (105.0x speedup), and the three-member replica set delivers resilient high availability with zero lost acknowledged writes during leader elections.

A three-phase investment roadmap is recommended for IoThings:

1. **Phase 1 (Correctness):** Implement the Transactional Outbox pattern to couple MongoDB writes atomically with MQTT actuator commands.
2. **Phase 2 (Hardening):** Migrate replica set nodes to isolated availability zones, enforce TLS encryption, and enable internal SCRAM-SHA-256 keyfile authentication ([MongoDB, Inc., 2026i](#)).
3. **Phase 3 (Scale-Out):** If write throughput eventually exceeds single-primary capacity, shard the collection using `{ device_id: "hashed", timestamp: 1 }` ([MongoDB, Inc., 2026g](#)) or migrate to MongoDB native Time Series collections ([MongoDB, Inc., 2026j](#)).

In conclusion, IoThings should retain relational databases for core billing and commercial accounts while deploying MongoDB as the dedicated enterprise platform for IoT telemetry.

References

- Abadi, D. (2012) ‘Consistency tradeoffs in modern distributed database system design: CAP is only part of the story’, *Computer*, 45(2), pp. 37–42.
- Brewer, E. (2012) ‘CAP twelve years later: how the “rules” have changed’, *Computer*, 45(2), pp. 23–29.
- Cattell, R. (2011) ‘Scalable SQL and NoSQL data stores’, *ACM SIGMOD Record*, 39(4), pp. 12–27.
- Chang, F. et al. (2008) ‘Bigtable: a distributed storage system for structured data’, *ACM Transactions on Computer Systems*, 26(2), pp. 1–26.
- Codd, E.F. (1970) ‘A relational model of data for large shared data banks’, *Communications of the ACM*, 13(6), pp. 377–387.
- DeCandia, G. et al. (2007) ‘Dynamo: Amazon’s highly available key-value store’, *ACM SIGOPS Operating Systems Review*, 41(6), pp. 205–220.
- Gilbert, S. and Lynch, N. (2002) ‘Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services’, *ACM SIGACT News*, 33(2), pp. 51–59.
- Information Commissioner’s Office (2026) *Storage limitation under UK GDPR*. Available at: <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/> (Accessed: 2 October 2026).
- Kleppmann, M. (2017) *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol: O’Reilly Media.
- Lakshman, A. and Malik, P. (2010) ‘Cassandra: a decentralized structured storage system’, *ACM SIGOPS Operating Systems Review*, 44(2), pp. 35–40.
- MongoDB, Inc. (2026a) *Indexes*. Available at: <https://www.mongodb.com/docs/manual/indexes/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026b) *Replication*. Available at: <https://www.mongodb.com/docs/manual/replication/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026c) *Replica set elections*. Available at: <https://www.mongodb.com/docs/manual/core/replica-set-elections/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026d) *Schema validation*. Available at: <https://www.mongodb.com/docs/manual/core/schema-validation/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026e) *Write concern*. Available at: <https://www.mongodb.com/docs/manual/reference/write-concern/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026f) *Transactions*. Available at: <https://www.mongodb.com/docs/manual/core/transactions/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026g) *Sharding*. Available at: <https://www.mongodb.com/docs/manual/sharding/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026h) *mongodump Database Tools*. Available at: <https://www.mongodb.com/docs/database-tools/mongodump/> (Accessed: 2 October 2026).

- MongoDB, Inc. (2026i) *Security checklist for self-managed deployments*. Available at: <https://www.mongodb.com/docs/manual/administration/security-checklist/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026j) *Time series collections*. Available at: <https://www.mongodb.com/docs/manual/core/timeseries-collections/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026k) *WiredTiger storage engine*. Available at: <https://www.mongodb.com/docs/manual/core/wiredtiger/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026l) *Expire data from collections by setting TTL*. Available at: <https://www.mongodb.com/docs/manual/tutorial/expire-data/> (Accessed: 2 October 2026).
- MongoDB, Inc. (2026m) *Read preference*. Available at: <https://www.mongodb.com/docs/manual/core/read-preference/> (Accessed: 2 October 2026).
- OASIS (2019) *MQTT Version 5.0, OASIS Standard*. Available at: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (Accessed: 2 October 2026).
- O’Neil, P. et al. (1996) ‘The log-structured merge-tree (LSM-tree)’, *Acta Informatica*, 33(4), pp. 351–385.
- OpenAPI Initiative (2020) *OpenAPI Specification v3.0.3*. Available at: <https://spec.openapis.org/oas/v3.0.3> (Accessed: 2 October 2026).
- PostgreSQL Global Development Group (2026) *PostgreSQL 18 documentation: JSON types*. Available at: <https://www.postgresql.org/docs/18/datatype-json.html> (Accessed: 2 October 2026).
- Robinson, I., Webber, J. and Eifrem, E. (2015) *Graph Databases: New Opportunities for Connected Data*. 2nd edn. Sebastopol: O’Reilly Media.
- Sadalage, P.J. and Fowler, M. (2012) *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Upper Saddle River, NJ: Addison-Wesley.
- Stonebraker, M. (2010) ‘SQL databases v. NoSQL databases’, *Communications of the ACM*, 53(4), pp. 10–11.

A Dataset Provenance and Sample BSON Document

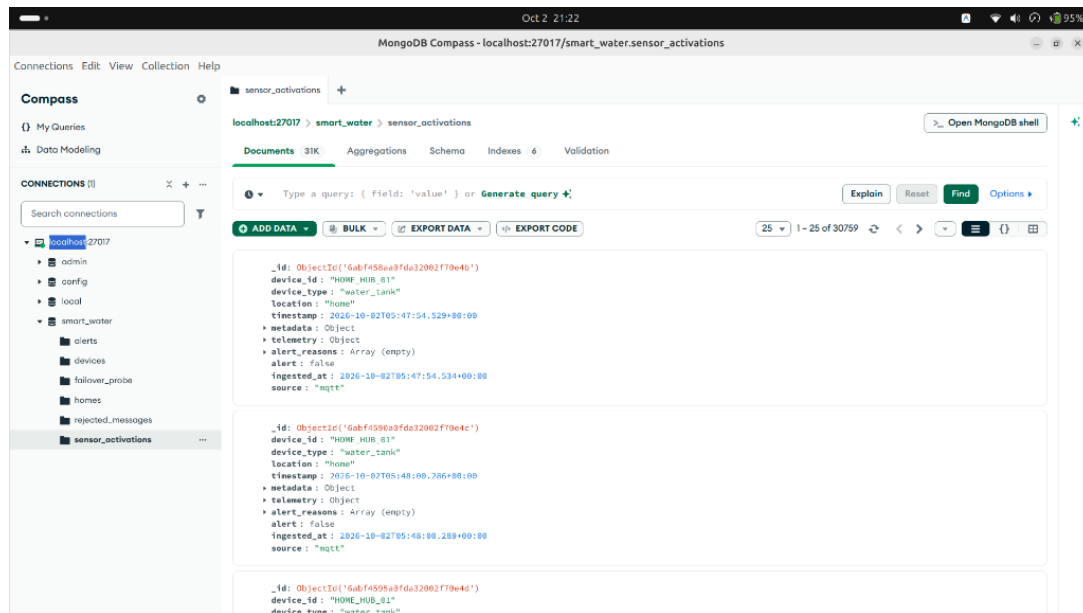


Figure 15: MongoDB Compass document explorer showing real-time BSON data structure.

B Local Replica Set Deployment Commands

```

1 # Terminal 1: Node 1 (Primary candidate)
2 mongod --replSet rs0 --port 27017 --dbpath ./mongo-cluster/node1 --bind_ip
  localhost
3
4 # Terminal 2: Node 2 (Secondary)
5 mongod --replSet rs0 --port 27018 --dbpath ./mongo-cluster/node2 --bind_ip
  localhost
6
7 # Terminal 3: Node 3 (Secondary)
8 mongod --replSet rs0 --port 27019 --dbpath ./mongo-cluster/node3 --bind_ip
  localhost
9
10 # Initialize replica set once in mongosh
11 rs.initiate({
12   _id: "rs0",
13   members: [
14     { _id: 0, host: "localhost:27017", priority: 2 },
15     { _id: 1, host: "localhost:27018", priority: 1 },
16     { _id: 2, host: "localhost:27019", priority: 1 }
17   ]
18 });

```

C Verification and Test Suite Status

The automated test suite was executed via `npm test (src/test.js)`, achieving **21 passed tests and 0 failures**:

- **Schema Validation & DLQ:** Correctly rejects invalid timestamps and malformed payloads, directing failures to the dead-letter queue.
- **Closed-Loop Hysteresis:** Dual-threshold hysteresis prevents relay chattering on both inlet valve and booster pump.
- **Dry-Run & Overflow Safety:** Automated threshold rules trigger alarms at $\leq 25\%$ and $\geq 85\%$ fluid levels.
- **Idempotent Messaging:** Unique compound index (`device_id`, `timestamp`) guarantees deduplication on duplicate MQTT QoS 1 frames.
- **Full CRUD Capability:** Verified across `homes`, `devices`, and `sensor_activations` with API key authentication.
- **Aggregation Pipelines:** Validates `$dateTrunc` and `$setWindowFields` calculations for non-refill volume drops.
- **Leak Engine Heuristics:** Verifies sensor noise tolerance, overnight slow-leak detection, and alert spam cooldown.
- **Depletion Prediction:** Validates active drain rate projections and graceful degradation on sparse data.
- **Distributed Cluster Health:** Confirms 3-node replica set connectivity, majority write concerns, and `secondaryPreferred` read routing.